Integración de Servicios Cloud GCP

De una idea a una webapp: frontend, backend, datos y archivos

Semana 11 - Sesión 21

Rel Guzman



Revisión: los bloques que ya construiste

Laboratorio	Lo que hiciste	El bloque de una webapp
Lab 01	Creaste buckets, subiste objetos y usaste Cloud Shell	**Archivos** y una forma de administrar recursos
Lab 02	Implementaste una pila LAMP desde Marketplace	**Servidor web, PHP y base de datos** en una VM
Lab 03 y 04	Probaste VPC, una VM privada, Acceso privado y Cloud NAT	**Red** y salida segura de los servidores
Lab 05 y 06	Configuraste VMs, discos, firewall, copias y scripts	**Cómputo**, datos persistentes y operación

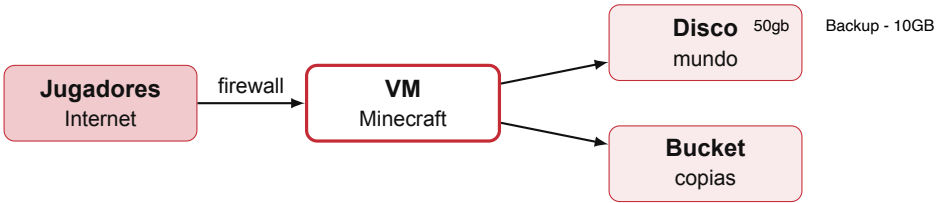
La pregunta de hoy

Cuando una aplicación necesita varios bloques, ¿cuál va en cada lugar y por qué?

Revisión: una aplicación que sí terminaste

El servidor de Minecraft del Lab 06

No era solo una VM. Tenía una IP externa para los jugadores, una regla de firewall para el puerto `25565`, un disco para el mundo, copias en un bucket y scripts para iniciar y detener el servicio.



Pregunta rápida

Si guardas el mundo solo dentro de la VM, ¿qué riesgo corres cuando la borras o su disco falla?

Logro de la sesión

Al finalizar, podrás reconocer las partes básicas de una webapp, explicar qué hace cada una y proponer una arquitectura inicial en Google Cloud que conecte **frontend**, **backend**, **base de datos**, ~~**archivos**~~ y ~~**seguridad**~~.

almacenamiento

Saberes previos

- Cuando abres una tienda en línea, ¿qué ves primero: el sitio, la base de datos o el servidor?
- Si una persona sube una foto de perfil, ¿debería quedar en la base de datos o como archivo?
- En una aplicación con usuarios y pedidos, ¿qué información no debería ser pública?

Utilidad de esta sesión

En un proyecto real no eliges un servicio por su nombre. Primero identificas **qué necesita la aplicación**: mostrar una página, ejecutar una regla, guardar datos, conservar archivos o proteger una clave. Luego eliges el servicio que cumple esa función.

Plan de hoy: dos partes y una práctica

Parte	Qué vemos	Lo conectamos con
1. Teoría	Las cinco piezas de una webapp	La tienda, la galería y el servidor de Minecraft
2. Teoría	Cuándo usar un servicio gestionado o una VM	Lo que ya configuraste en los labs
Práctica	Diseñar la arquitectura inicial de un inventario	El caso del curso y tus decisiones

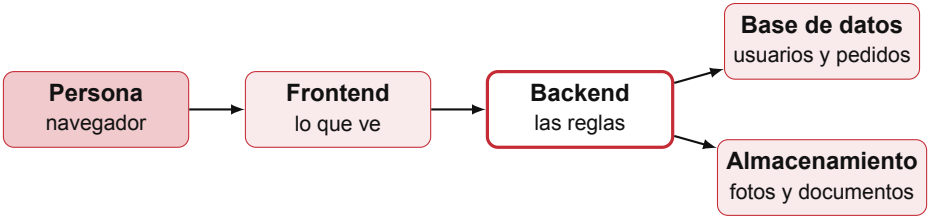
Regla de hoy

Una aplicación no tiene que usar todos los servicios. Solo usa los bloques que realmente necesita.

Parte 1: las piezas de una webapp

Del navegador a los datos y archivos

Una webapp en cinco piezas



Pieza	Su trabajo
Frontend	Muestra páginas, botones e imágenes en el navegador.
Backend	Recibe pedidos, valida reglas y conversa con los demás servicios.
Datos y archivos	Guardan información que debe sobrevivir cuando la aplicación se reinicia.

Frontend: lo que llega al navegador

Qué contiene

HTML, CSS, JavaScript, imágenes, íconos y fuentes. Es la parte visible de una landing, un catálogo o un panel.

Caso simple

Si las páginas ya están listas antes de la visita, puedes publicarlas como sitio estático. El navegador descarga los archivos y los muestra.

En Google Cloud

Un bucket puede guardar objetos. Para un sitio web público debes decidir explícitamente qué contenido será público y qué contenido se mantendrá privado.

Recuerda el Lab 01

Ya creaste un bucket y copiaste archivos desde Cloud Shell. El mismo bloque sirve para conservar archivos de una aplicación.

Backend: donde viven las reglas de la aplicación

Qué hace

Una API o backend recibe una petición, comprueba permisos, aplica una regla y devuelve una respuesta. Por ejemplo: “registrar pedido”, “calcular total” o “mostrar mis facturas”.

Cuando una VM tiene sentido

Necesitas controlar el sistema operativo, instalar software propio o mantener un proceso encendido. El servidor de Minecraft y la pila LAMP fueron ejemplos de este caso.

Cuando conviene que lo gestione el proveedor

Para una API pequeña o un contenedor, un servicio gestionado evita administrar parches, arranque y escalado de servidores.

La decisión no cambia la función

En ambos casos el backend sigue siendo el lugar donde viven las reglas. Lo que cambia es quién administra la infraestructura.

Base de datos: información que la aplicación consulta

Si guardas…	Una opción básica	Ejemplo
Tablas relacionadas	Base de datos relacional o SQL	Usuarios, productos, pedidos y facturas
Documentos flexibles	Base de datos de documentos o NoSQL	Perfiles con campos que cambian según el usuario
Datos temporales	Caché de clave y valor	Sesiones, contadores o resultados rápidos

No confundas dato con archivo

Un pedido puede guardar el nombre de una factura y su enlace. El PDF completo es un archivo, no una fila de texto.

Conexión con el Lab 02

MariaDB de LAMP era una base relacional dentro de la VM. Una base gestionada mantiene la misma función, pero el proveedor administra el servidor.

Archivos y secretos: no todo se publica

Recurso	Dónde va	Regla básica
Foto de un producto	Bucket público o entrega pública	Cualquiera puede verla si forma parte del catálogo
Factura o CV	Bucket privado y enlace temporal	El backend revisa quién puede descargarlo
Copia de seguridad y registros	Bucket privado separado	No se exponen desde la aplicación
Clave de API o archivo `.env`	Gestor de secretos	Nunca en el repositorio ni en un bucket

La idea del Lab 06

El mundo de Minecraft vivía en un disco y sus copias iban a un bucket. Separar la aplicación de su copia reduce el riesgo de perder ambos a la vez.

Parte 2: elegir dónde corre cada pieza

Servicio gestionado, VM y seguridad

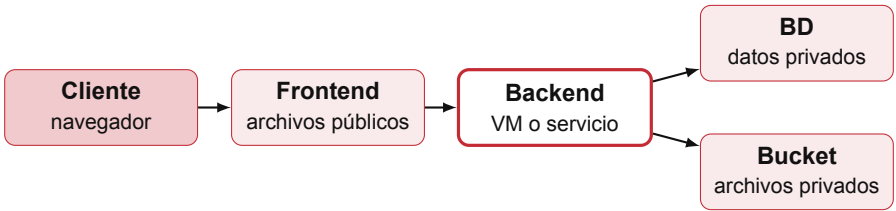
Servicio gestionado o VM

Pregunta	Servicio gestionado	Máquina virtual
¿Quién aplica parches?	El proveedor se encarga del servicio	Tú actualizas el sistema operativo y el software
¿Cuándo encaja?	Necesitas una función común: API, contenedor, base de datos o cola	Necesitas control del sistema, software especial o un proceso permanente
¿Qué debes vigilar?	Configuración, permisos y costos	Lo anterior, más disco, firewall, copias y mantenimiento

No hay una respuesta única

Una webapp puede usar frontend y base de datos gestionados, pero conservar una VM para un servicio que debe estar encendido todo el tiempo.

Una arquitectura inicial en Google Cloud



Lo que ya sabes proteger

La VPC organiza la red; el firewall decide qué tráfico entra; Cloud NAT permite que una VM privada salga a Internet para actualizarse.

Lo que no debe cambiar

El backend debe conservar acceso solo a los datos y archivos que necesita. Las claves van en un gestor de secretos.

Caso: inventario de una tienda

La necesidad

La tienda muestra productos, permite iniciar sesión, genera reportes, guarda facturas de proveedores y necesita respaldos. El equipo también quiere conservar las claves fuera del código.

Elemento	Decisión inicial	Razón
Catálogo y fotos de producto	Frontend y archivos de lectura pública	Los clientes deben verlos
Usuarios, productos y pedidos	Base de datos relacional	Hay relaciones entre las tablas
Facturas y reportes	Bucket privado con entrega controlada	Solo personal autorizado los consulta
Claves de producción	Gestor de secretos	No viajan con el código
Respaldos y registros	Bucket privado separado	No forman parte del sitio público

Práctica: dibuja una arquitectura inicial

En grupos de dos

Elijan una galería de fotos, una plataforma de entregas o una biblioteca de grabaciones. Dibujen solo los bloques necesarios y las flechas importantes.

Decidan	Pregunta que deben responder	Evidencia de los labs
Frontend	¿Qué ve el visitante?	Bucket y objetos del Lab 01
Backend	¿Qué regla ejecuta la aplicación?	LAMP o VM de los Labs 02, 05 y 06
Datos y archivos	¿Qué queda privado? ¿Qué necesita copia?	Disco, bucket y backup del Lab 06
Red y acceso	¿Qué puerto se abre? ¿Qué queda privado?	VPC, firewall y NAT de los Labs 03 y 04

Entregable

Un diagrama con al menos cuatro bloques y una frase que justifique cada decisión importante.

Cierre: preguntas para comprobar

Responde con tus palabras

1. ¿Qué hace el frontend y qué hace el backend?
2. ¿Por qué una factura PDF no debe guardarse igual que un pedido?
3. ¿Qué diferencia hay entre un bucket público y uno privado?
4. ¿Cuándo una VM tiene más sentido que un servicio gestionado?
5. En la tienda, ¿qué elementos dejarías sin IP externa y por qué?
6. ¿Qué laboratorio usarías como ejemplo para explicar copias de seguridad?

Idea final

Una arquitectura comienza al separar responsabilidades: interfaz, reglas, datos, archivos y seguridad.

Referencias

- Material base de esta sesión: `/Users/rgap/Desktop/Notas_Clases_GCP`: diagramas de frontend, API/backend, bases de datos, almacenamiento y otras cargas.
- Casos de práctica: `Notas_Clases_GCP/niveles/niveles.md`.
- Evidencias de laboratorios: `/Users/rgap/Desktop/Modelos_labs/Lab01` a `Lab06`.
- Material previo del curso: Sesiones 19 y 20, redes VPC, Acceso privado a Google y Cloud NAT.
- Google Cloud: Cloud Storage, Compute Engine, VPC, Cloud NAT, Cloud SQL, Firestore y Secret Manager.
- Las respuestas de las preguntas y de la práctica están en `../respuestas.md`.